

Reviewer Pack — Minimal Symplectic Volume and Communication Complexity Rank ...

Entry 46d5661b255a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 22:03:15 UTC

Minimal Symplectic Volume and Communication Complexity Rank Correlation

Entry ID: 46d5661b255a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 22:03:15 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every d -dimensional symplectic manifold M , the minimal symplectic volume ($\text{vol}_m(M)$) of M is linearly correlated with its communication complexity rank $r(M)$, such that $\text{vol}_m(M) = \Omega(r(M))$. For all constant $a > 0$ and d -regular graphs G , there exists a positive constant b such that $\text{vol}_m(G) \geq a * \log^{d(2)} * r(G)$.

Rationale (proposer's reasoning):

Symplectic geometry, which studies symplectic manifolds, provides an algebraic structure that may encode non-trivial geometric properties. The communication complexity of a graph can be interpreted as a measure of the difficulty in communicating information about the graph's structure. This conjecture suggests that symplectic volume, a geometric invariant, and communication complexity rank, a complexity-theoretic invariant, are related in a way that could reveal new structural insights.

Taxonomy category: symplectic_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6ddc47b6c9b6a2e1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the ratio of minimal symplectic volume to communication complexity rank must be greater than or equal to a threshold of $a * \log^{d(2)}$ for all graphs G and at least 80% of the seeds; to falsify it, this ratio must be less than the threshold for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            raise ValueError("Graph size must be a multiple of the degree")
        graph = {i: [] for i in range(n)}
        edges = set()
        nodes = list(range(n))

        while len(edges) < (n * d) // 2:
            node1, node2 = random.sample(nodes, 2)
            if (node1, node2) not in edges and (node2, node1) not in edges:
                graph[node1].append(node2)
                graph[node2].append(node1)
                edges.add((node1, node2))

        return graph

    def communication_complexity_rank(graph):
        n = len(graph)
        rank = 0
        for i in range(n):
            rank += len(graph[i])
        return rank

    def minimal_symplectic_volume(graph):
        n = len(graph)
        vol_m_G = 0
```

```

    for node in graph:
        for neighbor in graph[node]:
            if (node, neighbor) not in edges and (neighbor, node) not in edges:
                edges.add((node, neighbor))
                vol_m_G += 1
    return vol_m_G

a = Fraction(1, 2)
d = random.randint(3, 5)

results = []
for n in range(5, 41):
    graph = generate_d_regular_graph(n, d)
    vol_m_G = minimal_symplectic_volume(graph)
    r_G = communication_complexity_rank(graph)

    if r_G == 0:
        continue

    ratio = Fraction(vol_m_G, r_G)
    results.append(ratio)

metric_value = sum(results) / len(results)
conjecture_holds = all(ratio >= a * math.log(d, 2) for ratio in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "vol_m(G)/r(G)",
    "metric_value": float(metric_value),
    "instances_tested": len(results),
    "n_max": max(range(5, 41)),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    support_fraction = sum(result >= a * math.log(d, 2) for result in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={math.sqrt(sum((x - mean)**2 for x in results) / len(results))}")
    elif any(result < a * math.log(d, 2) for result in results):
        first_failing_seed = seeds[results.index(min(results))]
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80cd04fd.py", line 88, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80cd04fd.py", line 59, in run_trial
    graph = generate_d_regular_graph(n, d)
           ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80cd04fd.py", line 23, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
```

ValueError: Graph size must be a multiple of the degree

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with corrected input parameters that ensure the graph size is a multiple of the degree, and verify if the conjecture holds.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15286
2	preregistration	ollama_remote	glm4:latest	0	0	9700
3	novelty	ollama_remote	glm4:latest	0	0	8821
4	novelty	ollama_remote	glm4:latest	0	0	8665
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44525
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12207
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10597
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9211
9	judge	ollama_remote	glm4:latest	0	0	13558

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132571 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/46d5661b255a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/46d5661b255a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/46d5661b255a.tar.gz` (if generated)